01

Software(computer programs, libraries, data, documentation)

Software type(system, application)

Application software(stand-alone, web, phone)

System software(operation system, device drivers, utilities)

Generic software(general market, owned by development org), custom software(specific needs, owned by customer org)

Moodle – QMPLUS

Good software (required functionality, usable, efficient, dependable, maintainable, secure)

Software engineering layers: tool, method, process, quality focus (top and down)

Issues that affect software(heterogeneity, business/social change, security, scale, critical delivery)

02

Software process(requirement specification, development [analysis, design, implementation], validation[testing, deployment], evolution)

Requirement specification (what? Problems and constraints)

Analysis (conceptual model)

Design (implementable model (sufficient and detailed info), decompose components)

Implementation (coding, related document)

Testing (unit, functional, integration, system, acceptance) run in parallel with implementation

Deployment (live testing, training)

Evolution (reponse to customer needs) corrective, adaptive, perfective

Waterfall model -- sequential approach (appropriate when requirement well understood or limited change; enhancement of existing system; safety critical system ----traffic system)

Evolutionary development -- concurrent, interleaved activities, rapid feedback, many versions (appropriate when small size interactive system; short-lifetime system; multiple features --- social apps)

RUP -- iteration process (inception, elaboration, construction, transition)

03

Iteration (specification, design, implementation, testing are concurrent; rough specification, minimise documentation)

Agile process (iteration, incremental develop, involve end users) focus on code, deliver quickly

Agile focus on developers

Agile principle (customer involve, incremental delivery, embrace change, maintain simple, people)

Planing -- emphasize steer (direction)

Requirement -- user stories

XP features

Pair program (ad: share knowledge, learn from each other, review process, check error, help refactor; dis: conflicts, disagree, in different skill level)

Agile problems: (keep interest of customer, difficult to prioritize changes, extra work to maintain simple, unsuited to intense involvement)

Agile suits for small-medium, keep changing requirement, rapid deliver, involve customer, no

need document

04
Requirement: what system should do
Customer != end users
Functional vs non-functional requirement (system properties, process requirement)
Non-functional: product(efficiency, reliability, portability, usability), organisation(delivery, implementation, standard), external(ethical, legislative...)
Requirement measure: speed, size, ease of use, reliability, robustness, portability
Deal with conflicts: list options, select best one and take it to be agreed by stakeholders
Fact-finding technique (background reading, interview, observation, document or record sampling, questionnaires)

05
epics: large user stories
acceptance criteria: check the story more detailed and complete
All stakeholders write user stories
product backlog: a prioritised list of the functionality to be developed in a product
MoSCoW: must, should, could, want to
Estimating: T-shirt sizing, ideal time, hours, story points
Story points: points higher, uncertaintiy increase
prototype: fidelity: low--paper and sketch, medium--limited but clickable, on computer, high: closest resemblance, has details and functionality

06
Analysis: precise understanding of requirements, by developers
Conceptual models: using UML class diagram
Basic stereotypes for analysis class:
Entity: store data, define operations
Boundary: model interaction
Control: perform tasks
Attributes: description of data item; Operation: specification of behaviour
Class relationship: association(bidirectional. has name, role, multiplicity) and inheritance (from generalisation to specialisation)
Analysis steps (5)

07
Design: how sth work, how to solve problems through software, kind of blueprint
Compare with analysis: what & how, class type and attributes & all details, string & String
Encapsulation: information hiding
Modularity: seperate functionality -- independent, interchangeable modules
Good design: less coupling, more cohesion
coupling: dependency between subsystem; cohesion: dependency inside subsystem
Refactoring: intend to imrpove non-functional attributes

advantage of OOD: easy maintenance, reusable objects, mapping from real world to system
Design steps (6)

08
implementation subsystem: components, interfaces and other subsystems
software build incrementally in manageable steps -- executable version --version control
Integration build plan: describe the sequence of build. in each build: the functionality, which part of implementation model are affected by the build
Association in OO: references, collections
reference: unidirectional
arraylist: ArrayList<String> myList = new ArrayList<>();
array: MyObject[] objects; numbers = new int[5]; int[] numbers = {1, 2, 3, 4, 5};

09
who test: for component level: developers, for integration level: independent testing team
testing goals:
1. validation testing to both developer and customer (test for every requirement / all system feature) success: the system operates as intended
2. defect testing (failure, error, defect)
success: perform incorrectly and expose defects
Testing policies: approach in selecting system tests.
Good test: ease to find error; not redundant; best of bread; not simple not complex
Testing process: unit testing, system testing, acceptance testing
Testing strategy: what, how, when, how to determine
Test cases: input, result, conditions
Test procedures: general into specific
Black-box testing: external view: partition, scenario-based, regression (test requirement)
White-box testing: basis bath testing (test logic)
Partition testing: choose equivalence partitions; choose boundary/mid values
Scenario-based testing: for system testing
Regression testing: create test cases at each build stage (incremental)
White-box testing: all independent paths have been excised at least once.
Cyclomatic Complexity: number of decision + 1 or path number
Testing steps: use white to do unit testing (build harness, check method behave, robustness), use black to do integration testing

10
assertTrue(5 > 3);
assertFalse(5 < 3);
assertNull(str);
assertNotNull(str);
assertEquals("Hello", "Hello");
assertArrayEquals(expectedArray, actualArray);

12

advantages of focusing on architecture: system analysis (non-functional), large-scale reuse, stakeholder communication

agility and architecture: early stage of agile: design architecture

"While agile methodologies may not initially prioritize extensive upfront architectural design, it doesn't mean that architecture is disregarded or considered unimportant."

reason: System Understanding, Collaboration and Alignment, Scalability and Flexibility, Technical Risk Mitigation, Continuous Improvement

client-server architecture: multiple tiers

RESTful architecture: stateless operations

layered architecture: presentation, business, data

13

Type of project plan: quality plan (procedure and standard), validation plan, management plan...

Milestones vs deliverables: recognisable end-points of a process activity (formal outputs, definite) vs project results delivered to customers

Project scheduling: estimate time and resources

Split projects into separate tasks -- organize tasks concurrently -- minimise task dependency

Charts: task chart, activity network, bar chart

Critical path: the minimum time to complete the project

People management factors: consistency, respect, inclusion, honesty

Group working: composition, cohesiveness, communications, organisation

Scrum master: arrage stand up meetings, track backlog, record decision, measure progress, communicate with customers

14

Definition of risk management (identify, establish, make plans)

3 typs of risks(project, product, business)

[project: schedule or resources

product: quality or performance of software

business: organisation]

Risk management(identification, analysis, planning, monitoring)

Avoidance strategies (7)

Contigencies: alternative plans

Agile risk management: continuous testing, frequent interaction with the customer

Definition of quality management: establish process and standard by seperate team

Agile quality management: take place at the same time as implementation, a member, early release

High quality software: easy to use, ease to modify, good structure code

"Good enough" software: balance between meeting demand and giving guarantees

Visible and invisible quality: invisible(quality of code) visible(quality of user experience/security)

Avoid poor performance: employ skilled programmers, use library software; use tools to analyse code execution; avoid complex iteraction

Software reliability: probability of operating without failure; hard to estimate and ensure

Security and resilience: not be damaged intentionally and unintentionally; ensure few weakness; can continue operating or recover quickly
Software standard: aid cooperation; save time; develop from experience; help new staff understand -- become out of date though -- review regularly, supported by tools

15
Decomposition: locality and modifiability
Helper method: help split code into parts, delcare private
Reuse: source and benefit
Difference between abstraction and generalisation
IS-A HAS-A
Aliasing: more than one variable may refer to an object
Garbage collection:
parameter variable: close when the method finish; local variable: close when the block reach the end; object: remain even after the method call

single responsibility principle(SRP): high cohesive software.
Every object has a single responsibility. 代码实现：把不同功能分成函数块或对象
dependency: degrees of linkage HAS-A ; IS-A
weak dependency:take object as argument, but not call methods
Association: HAS-A through instance
Aggregation: an object is made up of objects from other class
Composition: a form of aggregation: the objects joined together exist only to make the aggregation

Open-closed principle(OCP): open for extension, close for modification　ABSTRACTION
change to meet new need, but not require chaning original code of module
Form of abstraction:
delegation, generalisation, parameterisation, type parameterisation
general -- specialist; essentials
Inheritance to meet OCP:1.　add extra features, override the methods 2. the variable of a class type is set to refer to subclass type 这样原代码不用改 (Liskov substitution priciple: unexpected)
parameterisation to meet OCP: modify the method without chaning code, by passing different parameter objects as its argument

Don't repeat yourself (DRY) 运用了泛化：通常先写具体，再寻找到 repeat 之后将其通过泛化转成更抽象的代码 变相 OCP
interface: only say WHAT
actural vs apparent: class of constructor vs type of the variable

Liscov substitution principle(LSP)
methods should not be overridden: code behaves in an unexpected way if we don't realize the variable is actually set to refer to an object of a subclass
not to use inheritance, but to use interface

Interface segergation principle(ISP)
importance of small interface types (have few methods)
new class only has to implement several which is relevant to

polymorphism and dynamic binding
upcasting is automated, while downcasting is mannual 只有向下转型后才可以调用子类的特有方法
instanceof break DIP
方法重写（同名）--调用子类方法
indirect subclass: 子类的子类=父类的子类

Dependency-inversion principle(DIP)
high and low level modules depend on abstractions
details should depend on abstractions
use interface types for parameters to methods and variables in class

16
design pattern: add addtional tasks to existing method; fit a class into an interface

Wrapper design pattern: outer object refer to inner object:
decorator design pattern: interface, concrete class, decorator class
优点： flexibility and modularity SRP, not introduce hierachy, OCP, code extensible and maintainable DRY

adaptor design pattern
将一个类的接口转化成用户期望的接口
target interfact, adapter, adaptee

composite design pattern 一致处理单个对象和组合对象
a collection of objects of a type is wrapped
component, leaf, composite

deal with aliasing: pass an immutable view of an object, rather than direct reference (can't change)

observer design pattern: link: an action on an object of one of the classes is observed by objects of the other class. Most common use to connect objects which store data or perform actions to other objects which dela with input and output
seperate model from view

factory method OCP
work like constructor and return new objects (can be an interface type), hide the actual type
put a wrapper around an object
不暴露具体的实例化过程

the singleton design pattern 单例模式
if exist, return an existing object instead of a new object

the object pool design pattern 对象池
return an object that was created before, keep a list of objects
use when the objects returned are immutable
避免频繁创建和销毁对象，可以多次获取和释放对象

the strategy design pattern -- generalising code by turning an action into a parameter 根据不同的类别作为参数读取，从而引用不同的方法
act on function objects

state design pattern:
used when we want to change the way we represent a type of object dynamically
mutable set: elements can be added or removed
定义不同的状态类在上下文中进行切换

the bridge design pattern
将 abstraction 和 implementation 分离，利用 collection 代替 inheritance
分别创建抽象类和实现类的对象，将实现类对象传递给抽象类对象的构造函数

flyweight pattern
object has an immutable part which can be shared
通过共享减少内存

17
Open source: the source code is publicly available
OSS: software free of charge, free of legal restriction for using
Why free: hobby; freedom belief; support software development as effective; earn fame
How to make profit: installation; offer training; provide consultancy advice to businesses
developers are users -- report bugs and update -- not affect other part
Advantage of OS: no risk of paying unreasonable charges
Software freedom: run; change; redistribute; distribute modified one
fork: maintain a separate version, but OSS prefer just one standard product
Copyright: legislation giving protection to creator of an original work (only...but can permit to..., which should involve obligations)
Copyleft: guarantee the freedom of copy and use, has a free software license (requirement, extent), copy and adaption -- same license
Android claim? 1. not satisfy both open governance model and licensing 2. not make public its internal decision-making processes 3. not provide public information like meetings and decisions made 4. not permit trademark used for devices

18
Version control: roll back to see all changes (check-out, check-in, conflicts, merge)

Build: produce a complete working version, compile and link
Continuous integration: full testing is involved in every modification
Bug database: what; which test; status; who correct
Active bug; resolved bug; closed bug
debuggers: suspend; breakpoints
Globalisation vs Localisation (different lan and cultures vs translation for individual lan)
command line operation vs IDE
command: type directly and interact directly with operating system; get closer control over computer; by writing scripts; understand how computer work
IDE: point and click; automate routine tasks; less flexibility over use of tools